RAY – AI Automation Engineer Case Study

Benjamin Kariuki · benjaminleyian50@gmail.com · November 2026

Code repository	github.com/benjaminkariuki/ray-ai-automation-case-study
Task 1 – Google Sheet	docs.google.com/spreadsheets/d/1GwAt8R3_O-h6ivZ_CT_8yD6GMIdybyf2-YgscTQ9hRI
Task 1 – Run history	GitHub Actions: daily runs at 06:00 UTC
Task 1 – README	task1-job-outreach/README.md
Task 1 – Sample output	sample-output/latest (console log, JSON, CSV of the newest run)
Task 2 – Notion (public)	https://billowy-target-f39.notion.site/Executive-Travel-Planner-Benjamin-Kariuki-412d73ee564d807d82e8f40be8b6009c
Task 2 – README	task2-travel-planner/README.md
Requirement checklist	REQUIREMENTS_CHECK.md: every line of the brief, how it's met, where to verify it

Task 1 – Executive Assistant Job Outreach Automation

What I built

A **Python pipeline**, run **every day at 06:00 UTC by GitHub Actions**, so no server of our own is needed. Each run:

- **Searches three job boards** (Himalayas, RemoteOK and Jobicy) through their official public APIs. LinkedIn is not used.
- **Checks each new posting with an LLM**: is it remote, English-speaking and relevant to an Executive or Admin Assistant? A job must pass all three to be kept.
- **Writes a personalised outreach email** for each match.
- **Fills a Google Sheet** that sales reps use as their dashboard.

Search terms, filters, AI models and prompts are all edited **in the Sheet**, so tone can be tested without touching code.

```
GitHub Actions (daily 06:00 UTC, or "Run workflow")  →  python -m job_outreach
  1  Read Settings + Prompts + already-processed URLs from the Google Sheet
  2  Fetch: Himalayas · RemoteOK · Jobicy   →  every posting gets title, company, location, description, URL, languages
  3  Rule filter: title keywords · excluded companies/agencies · max age · duplicates        (no AI cost)
  4  AI check per new job  →  JSON {remote, English, relevant, agency, languages, reason}
  5  AI outreach email for each job that passes all three
  6  Write Leads + Rejected, then the Run Log row  ·  save JSON / CSV / console log  ·  verify the live Sheet
  AI: OpenRouter (primary)  ──fails or hits its limit──►  Cloudflare Workers AI (backup)  ──both fail──►  retried next run
```

Requirements, as written in the brief

Brief	How it's met
✓ Compliant, non-LinkedIn job board	3 official public APIs, with no HTML scraping. Each row keeps the original URL and source name (the boards ask for a link back), and requests are paced.
✓ Collect title, company, location, description, URL and language requirements for each posting	All six fields for **every** scraped posting. Language requirements come from rules for every posting and are refined by the AI for checked jobs.
✓ Store in a structured format	Each run saves **JSON** (every posting and AI decision), **CSV** (all postings, leads, rejections) and a console **log**, plus the Google Sheet.
✓ Remote + English + relevant; keep only postings meeting all three	The LLM returns strict JSON with a reason. Only all-three passes become leads; the rest go to the **Rejected** tab with Yes/No per check. Staffing and VA agencies (competitors) are also excluded, which is configurable.
✓ Genuinely personalised message per posting	Each message cites duties, tools or details from that posting. Facts about us come only from the Sheet's "About us" setting, so nothing is invented.
✓ Easily interchangeable prompt	Prompts live in the Sheet's **Prompts** tab, and changing `Settings → outreach_prompt` switches them. Each lead records **Prompt Used**; both `v1_warm` and `v2_direct` were tested live.
✓ Google Sheet columns	The 6 required columns come first, in order, then Language, Source, dates, Prompt Used, AI Model, **Status** (a dropdown) and Notes.

✓ Daily or weekly schedule	GitHub Actions, daily at 06:00 UTC; one edit makes it weekly. A "Run workflow" button runs it on demand.
✓ README: install, run, filters and prompt, troubleshooting, sales rep guide	All covered, plus a 5-minute evaluator guide. A short guide is also built into the Sheet.

Deliverables

Deliverable	Where
Source code: scrape, validate, LLM, Sheets	`sources.py` · `ai.py` + `prompts/` · `ai.py` · `sheet.py` + `setup_sheet.py`, in `task1-job-outreach/`
Sample data output (CSV/JSON or console)	All three, in `sample-output/latest/`, refreshed automatically after every run
Scheduling setup	`.github/workflows/job-outreach.yml`, documented in README section 5
README	`task1-job-outreach/README.md`
Populated Google Sheet	The link on page 1: leads, rejections with reasons, and run history

Results and testing

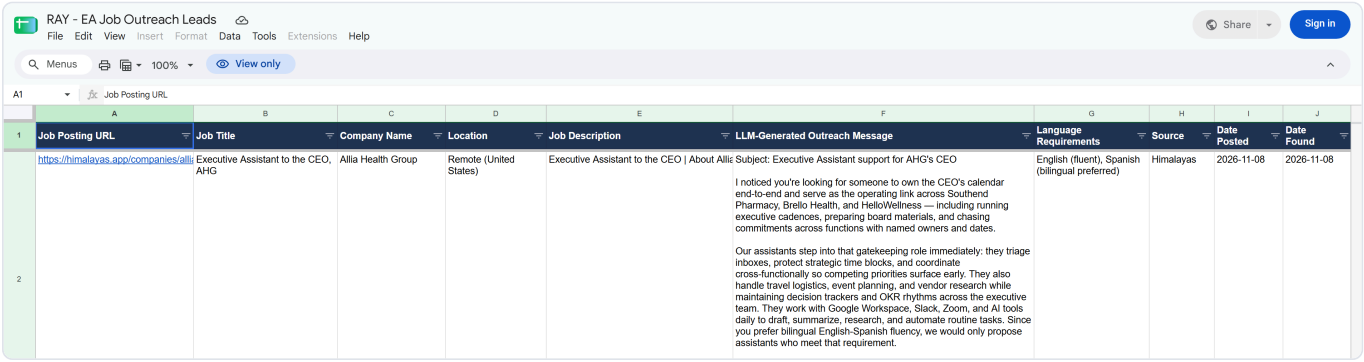
~340
postings fetched per run (3 boards)

37
leads written, each with a personalised email

36
rejections, each with a reason

12/12
live Sheet checks pass after every run

- **12 offline unit tests** (filtering, language detection, parsing, formatting, AI failover) run before every scheduled run.
- `verify_sheet.py` checks the live Sheet after every run: columns, no blanks, no duplicates, sources, personalised and unique messages, rejection logic, freshness. It needs no login, so anyone can run it.
- **Tested live:**
  - OpenRouter's limit ran out mid-test, and Cloudflare took over with 0 errors.
  - The prompt was swapped from the Sheet.
  - Duplicates were skipped.
  - Dry run.
- **Problems found and fixed during testing:**
  - The model invented details (a language skill, tool expertise).
  - An incomplete AI answer caused a false rejection; it's now retried instead.
  - "Must work Eastern Time hours" was wrongly judged non-remote.



The Leads tab: the six required columns first, then the AI-written message, language requirements, source, prompt used, AI model and a Status dropdown for the sales rep.

How to verify Task 1 in 5 minutes

1. Open the Sheet. In **Leads**, compare 2–3 messages with their Job Posting URL; each is specific to its posting.
2. **Rejected** shows why each job was dropped. **Prompts** and **Settings** show what's configurable. **Run Log** shows the daily runs.
3. In the repo, open `task1-job-outreach/sample-output/latest/` for the newest run's console log, JSON and CSV.
4. Optional: run `python task1-job-outreach/scripts/verify_sheet.py`; expect 12/12 PASS.

Process note: I first built this in n8n on a self-hosted server. It worked, and its testing drove the fixes above. I then moved it to Python on GitHub Actions so nothing depends on a server staying alive during the evaluation.

Task 2 – Executive Travel Planner (Notion)

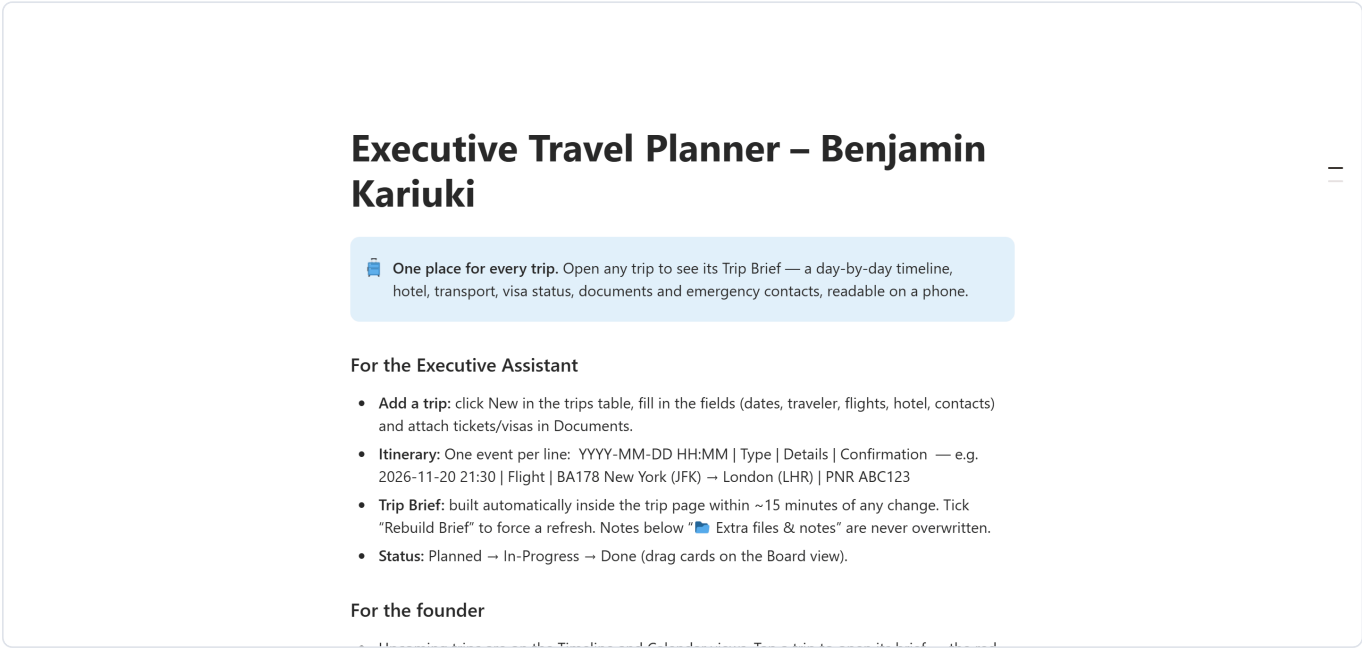
What I built

The page **"Executive Travel Planner – Benjamin Kariuki"** is published to the web, view-only. It has:

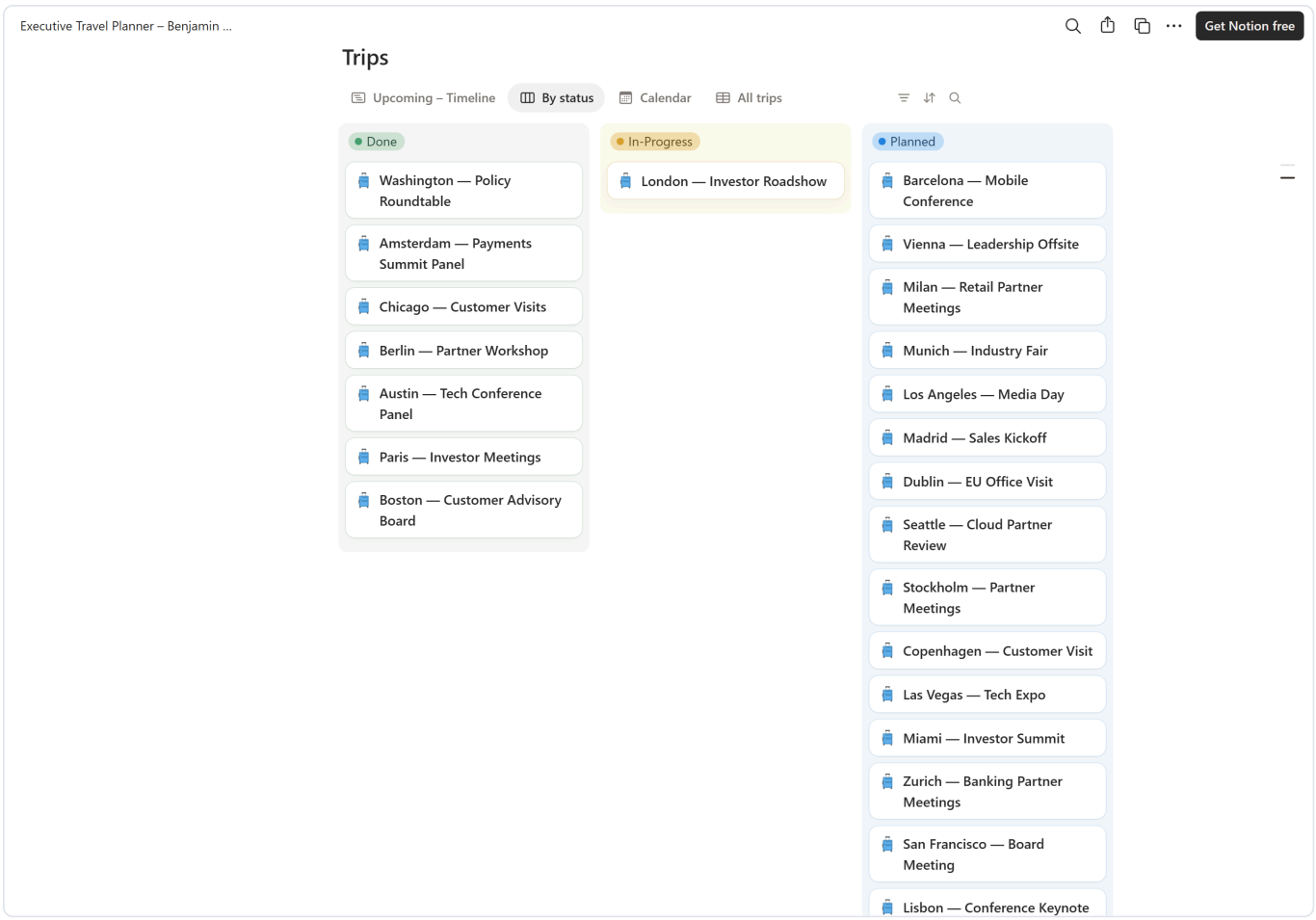
- A short **guide** for the EA and the founder.
- **One Trips database**, populated with **23 sample trips to Europe and the USA**: 7 Done, 1 In-Progress, 15 Planned.
- **4 saved views.**
- An **auto-generated Trip Brief** inside every trip.

The workspace was built with the Notion API (Python), so it's reproducible with one command.

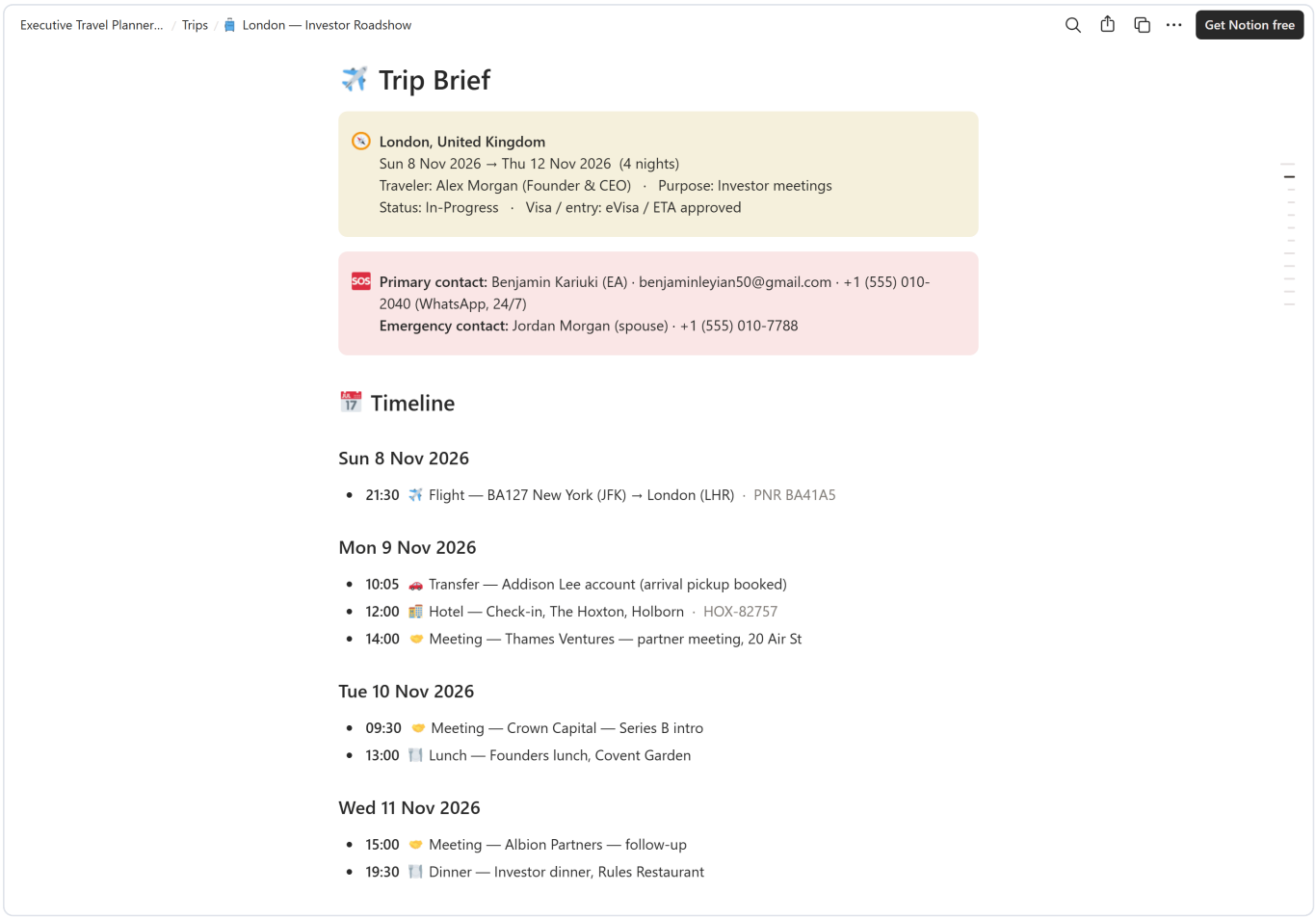
Brief	How it's met
✓ Single Trips database, one record per trip	One **Trips** database; every trip is one record.
✓ Trip name, who is traveling, date range	**Trip**, **Traveler**, **Travel Dates** (range), plus Status, Destination, Purpose and a **Countdown** formula ("15 days to go", "🛫 Travelling now").
✓ Flights, hotels, addresses, confirmations, meetings, notes	Flights (with PNR), Hotel, Hotel Address, Hotel Confirmation, Ground Transport, Visa / Entry and Visa Notes, an **Itinerary** with one line per event (meetings, transfers, dinners), and Notes.
✓ Trip Brief: auto-generated, shareable	A 🛫 **Trip Brief** in every trip page, written automatically from the trip's fields. It refreshes within 15 minutes of any edit (GitHub Actions + Notion API).
✓ Chronological timeline of key events	🗓️ **Timeline**: every event sorted by date and time and grouped by day, with icons (🛫 🚗 🏨 🤝 🍽️ 🎤) and confirmation numbers.
✓ Space to attach documents (tickets, visas)	The **Documents** files field (sample e-ticket, hotel confirmation and insurance PDFs attached), listed in the brief, plus a 📂 **Extra files & notes** area that's never overwritten.
✓ Emergency / primary contact info	A red 🆘 box near the top of every brief: primary contact (the EA), emergency contact and hotel.
✓ Timeline/calendar view of upcoming trips	**Upcoming – Timeline** (hides Done trips) and **Calendar**.
✓ Overview grouped by status	A **By status** board with columns Planned / In-Progress / Done.
✓ Share to web, titled "Executive Travel Planner – {Your Name}"	Published, view-only: "Executive Travel Planner – Benjamin Kariuki".



The planner page: a short guide for the EA and the founder, then the Trips database (Upcoming – Timeline view).



The By status view: Planned / In-Progress / Done.



A Trip Brief, generated automatically: summary, SOS contacts and the day-by-day timeline.

Design choices (the optional note)

- **One database, structured fields.** The EA fills in plain fields and never formats anything. The itinerary is one line per event (`2026-11-20 21:30 | Flight | BA178 JFK → LHR | PNR ABC123`), which is quick to type and easy to sort.
- **A generated brief instead of a static template.** A Notion template fills a page once and goes stale. Here the brief is rebuilt from the fields whenever the trip changes, so the founder never sees an outdated plan. Lines in the wrong format appear in a ⚠️ box rather than being lost.
- **Founder-first layout.** Where and when come first, then who to call, then the day-by-day plan, then the hotel with a Google Maps link, transport, visa status and documents. Only headings, bullets and callouts are used (no wide tables), so it reads well on a phone.
- **Status that drives the views.** Planned → In-Progress → Done moves cards on the board. The timeline hides finished trips, and the countdown shows what's next.
- **Tested.**
  - 4 unit tests.
  - Live build: 23 trips with 23 complete briefs, each checked for timeline, contacts, stay, visa and documents; 0 schedule conflicts.
  - Editing one trip rebuilds only its brief.
  - EA notes survive rebuilds.